

9 – FILE I/O & ERRORS

9.a – Learning Outcomes

By the end of this chapter, you will be able to:

1. Explain why file I/O is essential for persisting data beyond a program's execution.
2. Open files using the `open()` function and describe the purpose of each parameter.
3. Differentiate between file modes ('r', 'w', 'a', 'x', 't', 'b') and select the appropriate mode for a given task.
4. Write text to a file using the `write()` method and ensure the file is properly closed.
5. Read file contents using `read()`, `readline()`, and `readlines()`, and explain the difference between each method.
6. Use the `with` statement to perform file I/O safely, explaining how it automatically manages resource cleanup.
7. Perform common file management operations using the `os` module, including checking file existence, deleting files, creating folders, and removing empty folders.
8. Identify and distinguish between common Python errors: `SyntaxError`, `NameError`, `TypeError`, `ValueError`, `IndexError`, `KeyError`, `FileNotFoundError`, and `ZeroDivisionError`.
9. Describe the purpose of `try`, `except`, `else`, and `finally` blocks in error handling.
10. Apply basic error handling techniques to write programs that respond gracefully to runtime errors.
11. Recognize the risks of using the `os` module for direct filesystem manipulation.

9.b – Introduction

Writing to a file is the primary way to **persist data** after a program finishes running. Without file I/O, all data created by a program is lost when the program terminates. In this course, we will work **only with text files** — everything written to or read from a file will be strings.

This chapter covers two essential topics:

- **File I/O** – reading from and writing to files on disk
- **Errors** – understanding what can go wrong and how to handle it

Understanding both topics is critical for building robust programs that can save their results and recover gracefully from unexpected situations.

Important: When working with files, you must always close the file after completing all input or output operations. Failure to do so can result in data loss or corruption. The `with`

statement (covered in section 9.a.4) is the preferred way to ensure files are closed automatically.

9.c – Working with files

Writing to file is the primary way to persist data after a program finish running. In this course we will work ONLY be using text files i.e. everything written or read from a file will be strings.

9.c.1 – Opening a file

The `open()` function opens a file and return a **stream** (file handle in other languages). This stream can be used to write to or read from a file. You must always close the stream after completing all input or output operations.

```
f = open(
    «filename»,          #pathname to the file
    [mode = 'r'],        #read/write mode
    [encoding = None],    #ignore this argument
    [newline = None]      #also ignore this argument
)
```

The file should always be closed after you finish using it.

9.c.2 – Writing to a file

The `write()` method sends string to the opened file.

```
f = open('test.txt', 'wt')      # write text mode
f.write('file created by Narendra')
f.close()                       #closes the file
```

9.c.3 – Reading from a file

Three commonly used methods:

Method	Description
<code>read()</code>	returns the entire file content as a single string.

<code>readline()</code>	returns the current line in the file as a single string and move the pointer to the next line. A subsequent read will get the next line.
<code>readlines()</code>	returns the entire file content as string items of a list.

```
f = open('test.txt')      #no mode is specified
first = f.readline()      #reads the first line
rest = f.read()           #read the rest of the file
f.close()                 #closes the file
```

9.c.4 – A more elegant way of reading/writing a file

The with statement is a more modern way to performing file IO.

```
with open('test.txt') as f:
    for line in f:
        print(line)        #automatically the file is closed and all
                            #associated resources released
```

The file closes automatically once processing move out of the with block.

9.c.5 – File Modes

Mode	Meaning
<code>'r'</code>	Read (default)
<code>'w'</code>	Write (overwrites file)
<code>'a'</code>	Append to file
<code>'x'</code>	Creates a new file
<code>'t'</code>	Text mode (default)
<code>'b'</code>	Binary mode

9.d – Operation System File Management

You will need to import the `os` module to access the file management capabilities of Python. Fortunately, this module does not need to be downloaded, because it is a part of the standard Python installation.

Be very careful when using this module, because you will have direct access to your computer filesystem and you have the ability to corrupt your machine.

9.d.1 – Checking for the existence of a file

You will use the `exist()` method to check if the path (either file or folder) is present.

```
>>> import os
>>> os.path.exists('person.dat')
True
>>> os.path.exists('foo.dat')
False
```

9.d.2 – Deleting a file

You will use the `remove()` method to delete a file.

```
>>> if os.path.exists('person.dat'):
    os.remove('person.dat')
```

9.d.3 – Deleting a folder

You will use the `rmdir()` method to delete a folder. Please be aware that you are only able to remove empty folders.

```
os.rmdir('temp')
```

9.d.4 – Creating a folder

You will use the `mkdir()` method to create a folder.

```
os.mkdir('foo')
```

9.e – Errors

Syntax errors happen when your code does not make any sense to the computer. In other words, it encounters something that it does not expect! This can happen because you've misspelled something or there is one too many brackets or a missing comma or even the incorrect indentation level

9.e.1 – Syntax Error

Occurs when Python encounters invalid syntax.

```
print(12 + 4))
File "<python-input-0>", line 1
    print(12 + 4))
                ^
SyntaxError: unmatched ')'
```

9.e.1 – Name Error

This happens when there is a variable with a name that the computer does not recognize. It's usually because you've misspelled the name of a variable you created earlier.

```
my_number = 4
my_Number + 2
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    my_Number + 2
    ^^^^^^^^^
NameError: name 'my_Number' is not defined. Did you mean: 'my_number'?
```

Remember variable names are case sensitive!

9.e.2 – Type Error

This happens when an operation is performed on an inappropriate type.

```
>> 'H' + 5
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
```

```
'H' + 5
~~~~^~~
```

TypeError: can only concatenate str (not "int") to str

9.e.3 – Value Error

This happens when a function receives an argument of correct type but inappropriate value.

```
>>> import math
>>> math.sqrt(-1)
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    math.sqrt(-1)
    ~~~~~^~~~~
ValueError: math domain error
```

9.e.4 – Index Error

This happens when a sequence subscript is out of range.

```
>>> a = [1, 2, 3, 4, 5]
>>> a[5]
Traceback (most recent call last):
  File "<python-input-4>", line 1, in <module>
    a[5]
    ~^^^
IndexError: list index out of range
```

9.e.5 – Key Error

This happens when a dictionary key is not found.

```
>>> a = {1: 'one', 2: 'two', 3: 'three'}
>>> a[0]
Traceback (most recent call last):
  File "<python-input-16>", line 1, in <module>
    a[0]
    ~^^^
KeyError: 0
```

9.e.6 – FileNotFoundError

This happens when a file or directory is requested but doesn't exist.

```
>>> f = open('missing.txt', 'rt')
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    f = open('missing.txt', 'rt')
FileNotFoundError: [Errno 2] No such file or directory: 'missing.txt'
```

9.e.7 – Zero Division Error

This happens when you try to divide by zero. This is something that is mathematically impossible so Python will also complain.

```
5 % 0
Traceback (most recent call
last): File '<stdin>', line 1,
ZeroDivisionError: integer division or modulo by zero
```

9.e.8 – Error handling

Error handling is an advanced topic and might not be a part of the course.

```
try:
    python statement that can cause an error situation

except «type_of_error1»:
    do something

except «type_of_error2»:
    do something
```

9.e.9 – A Practical Example

```
try:
    # Code that might cause an error
```

```
number = int(input("Enter a number: "))
result = 10 / number
print(f"Result: {result}")

except ValueError:
    # Handle invalid number conversion
    print("That's not a valid number!")

except ZeroDivisionError:
    # Handle division by zero
    print("Cannot divide by zero!")

except Exception as e:
    # Handle any other exception
    print(f"An error occurred: {e}")

else:
    # Runs if no exception occurred
    print("Operation successful!")

finally:
    # Always runs (cleanup code)
    print("Done!")
```

9.f – Summary

This chapter covered two essential topics for writing robust Python programs: file I/O and error handling.

9.f.1 – File I/O:

- `open()` creates a file stream. Always close the file when done.
- **File modes** control how the file is opened: `'r'` (read), `'w'` (write/overwrite), `'a'` (append), `'x'` (create new), `'t'` (text), `'b'` (binary).
- `write()` writes a string to a file.
- `read()`, `readline()`, and `readlines()` read file contents in different ways.

- The `with` statement is the preferred way to handle file I/O because it automatically closes the file, even if an error occurs.
- The `os` module provides file management operations: `exists()`, `remove()`, `rmdir()`, `mkdir()`. Use with caution.

9.f.2 – Errors:

- `SyntaxError` – invalid syntax (e.g., unmatched parentheses)
- `NameError` – undefined variable (often a misspelling or case error)
- `TypeError` – operation on an inappropriate type
- `ValueError` – correct type but inappropriate value
- `IndexError` – sequence index out of range
- `KeyError` – dictionary key not found
- `FileNotFoundError` – file or directory does not exist
- `ZeroDivisionError` – division by zero

9.f.3 – Error Handling:

- `try/except` blocks allow you to catch and handle exceptions gracefully.
- `else` runs if no exception occurred.
- `finally` always runs, making it ideal for cleanup code.
- Catch specific exceptions before general ones.

9.g – Non-Coding Exercises

1. What is the primary purpose of file I/O in a program?
2. What does the `open()` function return?
3. Why must you always close a file after using it?
4. What is the default mode for `open()`?
5. What is the difference between `'w'` and `'a'` file modes?
6. What does the `'x'` file mode do?
7. What is the difference between `read()`, `readline()`, and `readlines()`?
8. What happens if you call `readline()` after `read()`?
9. Why is the `with` statement preferred over manually calling `close()`?
10. What module must you import to perform file management operations?
11. What method checks if a file or folder exists?
12. What method deletes a file?
13. What method creates a folder?
14. What method deletes a folder? What restriction applies?
15. What is the difference between a syntax error and a runtime error?
16. What error occurs when you misspell a variable name?

17. What error occurs when you add a string and an integer?
18. What error occurs when you access an index that doesn't exist?
19. What error occurs when you access a dictionary key that doesn't exist?
20. What error occurs when you try to open a file that doesn't exist?
21. What error occurs when you divide by zero?
22. What is the purpose of the `try` block?
23. What is the purpose of the `except` block?
24. What is the purpose of the `else` block in a `try/except` structure?
25. What is the purpose of the `finally` block?
26. Why is it better to catch specific exceptions rather than a general `Exception`?
27. What happens if an exception occurs inside a `with` block? Is the file still closed?
28. Why should you be careful when using the `os` module?